

ITIL Ticket: vLLM Performance Optimization for Dell RTX 6000

Ticket ID: ITIL-TICKET-DLL-VLLM-PERFORMANCE **Created:** 2026-05-12 14:49 UTC **Priority:** High **Category:** Infrastructure Optimization **Assigned To:** Lavid (Jeremy Ingalls) **Status:** Open

Problem Statement

The vLLM service running on Dell's RTX Pro 6000 Blackwell (96GB VRAM) is experiencing sporadic and slow processing for parallel requests. The single GPU allocates all compute resources to every request, leaving nothing for concurrent processing. This impacts agent workflows that require large context windows (128K tokens) and large output sizes (5-15K tokens).

Current Configuration

Hardware

- **GPU:** NVIDIA RTX Pro 6000 Blackwell Workstation Edition
- **VRAM:** 96GB (97,887 MiB)
- **VRAM Usage:** 90.5GB (92.5%)
- **VRAM Free:** 6.7GB (6.7%)
- **GPU Utilization:** 0% (idle after test)
- **Power Draw:** 90.20W

Software

- **vLLM Version:** 0.20.2
- **Model:** Qwen3-Next-FP8 (74.86GB)
- **Context Length:** 128K tokens (131,072)
- **Tool Parser:** qwen3_coder

Current Args

```
--max-model-len=131072
--max-num-seqs=512
--enable-auto-tool-choice
--tool-call-parser=qwen3_coder
```

Missing Args (Critical for Concurrency)

- `--max-running-requests` (defaults to unlimited)
- `--max-num-batched-tokens` (defaults to unlimited)
- `--gpu-memory-utilization` (defaults to 0.9)
- `--enable-prefix-caching` (not enabled)
- `--max-prompt-len` (defaults to `max_model_len`)

Performance Test Results

Sequential Requests

- Request 1: 0.80s
- Request 2: 0.78s
- Request 3: 0.71s

Concurrent Requests (5 simultaneous)

- All 5 requests: ~1.56s each

Key Finding

Concurrent requests all completed in the same time (~1.56s), suggesting vLLM is batching them efficiently. However, latency doubled compared to sequential requests (0.8s → 1.56s), indicating the GPU is being shared but not optimally.

Root Cause Analysis

1. **GPU Memory Pressure:** 90.5GB/96GB used (94.3%) leaves only 5.7GB for KV cache
2. **No Concurrency Limits:** `--max-running-requests` not set, allowing unlimited concurrent requests
3. **No Batch Size Limits:** `--max-num-batched-tokens` not set, allowing unlimited batch sizes
4. **No Prefix Caching:** KV cache not reused for repeated prompts
5. **Large Context Length:** 128k tokens per request requires massive KV cache per request

Proposed Solutions

Option 1: Add Concurrency Limits (Non-Invasive)

```
--max-running-requests=4 --max-num-batched-tokens=2048 --gpu-memory-utilization=0.90
```

- **Pros:** Limits concurrent requests to 4, caps batch size to 2048 tokens, leaves 10% GPU memory buffer
- **Cons:** May clip large outputs (5-15K tokens)

Option 2: Enable Prefix Caching (Non-Invasive)

```
--enable-prefix-caching
```

- **Pros:** Reuses KV cache for repeated prompt prefixes, helps when multiple users send similar prompts
- **Cons:** Minimal impact on large context windows

Option 3: Quantize Model (Requires Rebuild)

- **VRAM Savings:** ~50% (75GB → ~37.5GB)

- **Quality Impact:** Minimal for most use cases
- **Rebuild Required:** Yes
- **vLLM Support:** Yes (`--load-format=int8`)

Option 4: Increase Batch Size

```
--max-running-requests=8 --max-num-batched-tokens=8192 --gpu-memory-utilization=0.90
```

- **Pros:** Allows 8 concurrent requests, 8192 token batch size (handles 10K token outputs in ~1-2 chunks)
- **Cons:** Less concurrency than Option 1

Option 5: Quantize + Increase Batch Size

```
--load-format=int8 --max-running-requests=8 --max-num-batched-tokens=8192 --gpu-memory-utilization=0.90 --enable-prefix-caching
```

- **Pros:** 50% VRAM savings (75GB → ~37.5GB), 37.5GB free for KV cache and concurrent requests, handles large context windows AND large outputs AND concurrency
- **Cons:** Requires model rebuild, ~10-20% slower than FP8

Recommendation

Option 5: Quantize + Increase Batch Size - Quantization: INT8 (`--load-format=int8`) - **Concurrency:** 8 concurrent requests - **Batch Size:** 8192 tokens - **GPU Memory:** 90% utilization - **Prefix Caching:** Enabled

This provides the best balance of: - Large context windows (128K tokens) - Large outputs (5-15K tokens) - Multiple concurrent agents - Minimal quality impact

Next Steps

1. **Review this ticket** with Lavid to confirm requirements
2. **Select preferred option** (Option 5 recommended)
3. **Implement changes** in the vLLM service
4. **Test performance** with concurrent requests
5. **Monitor GPU memory** and adjust as needed

Appendix

vLLM Quantization Support

- fp8 (current)
- fp8_e4m3
- fp8_e5m2
- int8
- int4
- awq
- gptq

- gptq_marlin
- marlin
- squeezellm
- compressed_tensors
- bitsandbytes
- weight_only

Model Architecture

- **Type:** Qwen3-Next (Mixture of Experts)
- **Size:** 75GB (40 shards)
- **Quantization:** FP8
- **Architecture:** 512 experts, 10 experts per token
- **Hidden Size:** 2048
- **Num Layers:** 48
- **Num Attention Heads:** 16
- **Num Key-Value Heads:** 2
- **Max Position Embeddings:** 262,144
- **Num Experts:** 512
- **Num Experts per Token:** 10
- **Moe Intermediate Size:** 512